

ДИСЦИПЛИНА	Технологии индустриального программирования
ИНСТИТУТ	Институт перспективных технологий и индустриального программирования
КАФЕДРА	Кафедра индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Практическое занятие
ПРЕПОДАВАТЕЛЬ	Адышкин Сергей Сергеевич
СЕМЕСТР	2 семестр, 2025-2026 гг.

Практическое занятие №8

Настройка GitHub Actions / GitLab CI для деплоя приложения

Цель занятия

Освоить основы CI/CD для backend-проекта на Go, научиться настраивать автоматический pipeline для проверки, сборки, упаковки Docker-образа и подготовки приложения к доставке.

Задачи занятия

1. Изучить назначение CI/CD в современной backend-разработке.
2. Понять различие между CI и CD.
3. Научиться создавать pipeline для автоматического запуска тестов и сборки.
4. Освоить базовую настройку GitHub Actions или GitLab CI.
5. Научиться собирать Docker-образ внутри CI.
6. Понять, как использовать секреты и переменные CI.
7. Освоить базовую публикацию Docker-образа в registry.
8. Разобрать логику минимального автоматизированного деплоя.

Теоретическая часть

Что такое CI

CI – это Continuous Integration, то есть непрерывная интеграция. Смысл в том, что после каждого изменения кода система автоматически проверяет проект:

- устанавливает зависимости;
- запускает тесты;
- выполняет сборку;
- проверяет, что код не сломал проект.

Это особенно важно в командной разработке, где изменения в репозиторий приходят регулярно.

Что такое CD

CD может трактоваться как:

- Continuous Delivery – готовность к доставке;
- Continuous Deployment – автоматическое развёртывание.

Для учебной практики достаточно понимать так:

CI отвечает за автоматическую проверку и сборку. **CD** отвечает за упаковку и доставку результата – например, публикацию Docker-образа и деплой на сервер.

Зачем backend-проекту pipeline

Pipeline нужен не для красоты и не только для крупных компаний. Даже в учебном проекте он полезен, потому что:

- исключает ручные забывания;
- автоматически показывает, сломался ли проект;

- формирует воспроизводимую сборку;
- готовит проект к деплою.

Если есть Docker-образ и CI, то проект уже ближе к реальной практике разработки.

Что обычно делает pipeline

В типичном backend-проекте pipeline может выполнять:

- checkout репозитория;
- настройку версии Go;
- загрузку зависимостей;
- запуск `go test ./...`;
- запуск `go build ./...`;
- сборку Docker-образа;
- публикацию образа в registry;
- деплой на сервер.

В учебной работе обязательной будет только базовая часть: тесты, сборка и Docker build.

Зачем нужны secrets

Pipeline может работать с токенами, ключами и паролями. Эти данные нельзя хранить прямо в репозитории.

Поэтому CI-системы предоставляют специальные хранилища:

- GitHub Secrets / Variables;
- GitLab CI/CD Variables.

Здесь важно понять принцип: секреты не коммитятся в репозиторий и не пишутся прямо в YAML-файл.

Что мы сделаем в этой работе

В данной практике будет создан минимальный pipeline для Go-проекта.

Вариант на GitHub Actions:

- файл `.github/workflows/ci.yml`

Вариант на GitLab CI:

- файл `.gitlab-ci.yml`

Pipeline будет:

- проверять код;
- запускать тесты;
- выполнять сборку;
- собирать Docker-образ.

Дополнительно будет показан опциональный сценарий публикации в registry и минимального деплоя на VPS.

Практическая часть

Общая идея проекта

В качестве основы используется уже существующий репозиторий с сервисом tasks или с двумя сервисами auth и tasks.

Если у вас два сервиса, допустимо сначала собрать pipeline только для одного, но в отчёте нужно пояснить, почему выбран именно он.

Шаг 1. Подготовка репозитория

Перед настройкой CI нужно убедиться, что проект:

- собирается локально;
- тесты запускаются локально;
- Dockerfile уже существует и работает.

Минимальная проверка:

```
go test ./...
go build ./...
docker build -t techip-tasks:0.1 .
```

Шаг 2. Вариант на GitHub Actions

Создайте файл:

```
.github/workflows/ci.yml
```

Содержимое файла:

```
name: CI Pipeline

on:
  push:
    branches: [ "main", "master" ]
  pull_request:
    branches: [ "main", "master" ]

jobs:
```

test-and-build:

runs-on: ubuntu-latest

steps:

- name: Checkout repository
uses: actions/checkout@v4

- name: Setup Go
uses: actions/setup-go@v5
with:
go-version: '1.23'

- name: Show Go version
run: go version

- name: Download dependencies
run: go mod tidy
working-directory: ./services/tasks

- name: Run tests
run: go test ./...
working-directory: ./services/tasks

- name: Build application
run: go build ./...
working-directory: ./services/tasks

docker-build:

runs-on: ubuntu-latest

needs: test-and-build

steps:

- name: Checkout repository
uses: actions/checkout@v4

- name: Set up Docker Buildx
uses: docker/setup-buildx-action@v3

- name: Build Docker image

```
run: docker build -t techip-tasks:${{ github.sha }} .  
working-directory: ./services/tasks
```

Пояснение к структуре pipeline

В этом pipeline есть два job:

test-and-build

- получает код;
- устанавливает Go;
- запускает тесты;
- выполняет сборку.

docker-build

- запускается только после успешного первого job;
- собирает Docker-образ.

Такое разделение делает pipeline более понятным.

Шаг 3. Проверка работы GitHub Actions

Сделайте commit и push в репозиторий.

После этого откройте вкладку **Actions** в GitHub.

Pipeline должен стартовать автоматически.

Если всё настроено корректно, оба job завершатся успешно.

Шаг 4. Вариант на GitLab CI

Если используется GitLab, создайте файл .gitlab-ci.yml:

```
stages:  
  - test  
  - build  
  - docker  
  
test_job:  
  stage: test
```



```
image: golang:1.23
script:
  - cd services/tasks
  - go mod tidy
  - go test ./...

build_job:
  stage: build
  image: golang:1.23
  script:
    - cd services/tasks
    - go build ./...

docker_job:
  stage: docker
  image: docker:latest
  services:
    - docker:dind
  script:
    - cd services/tasks
    - docker build -t techip-tasks:$CI_COMMIT_SHORT_SHA .
```

Шаг 5. Формирование тега Docker-образа

В CI важно не собирать безымянные образы.

Минимально допустимые варианты:

- latest;
- короткий hash коммита;
- одновременно latest и commit-based tag.

Пример:

- GitHub Actions: `{{ github.sha }}`
- GitLab CI: `$CI_COMMIT_SHORT_SHA`

Это позволяет понять, какая именно версия собрана.

Шаг 6. Работа с secrets

Если pipeline должен:

- публиковать образ в registry;
- подключаться по SSH к серверу;
- использовать внешний токен;

то нужно хранить такие данные в secrets.

Примеры:

- REGISTRY_USERNAME
- REGISTRY_PASSWORD
- SSH_PRIVATE_KEY

Их нельзя:

- коммитить в репозиторий;
- писать в .yaml в открытом виде;
- хранить в .env, который попадает в Git.

Шаг 7. Пример публикации образа в registry

В учебной работе публикация образа может быть опциональной, если у вас нет подготовленного registry.

Фрагмент для GitHub Actions может выглядеть так:

```
- name: Login to registry
  run: echo "${{ secrets.REGISTRY_PASSWORD }}" | docker login -u "${{ secrets.REGISTRY_USERNAME }}" --password-stdin ghcr.io

- name: Build image
  run: docker build -t ghcr.io/my-org/techip-tasks:${{ github.sha }} .
  working-directory: ./services/tasks

- name: Push image
  run: docker push ghcr.io/my-org/techip-tasks:${{ github.sha }}
```

Здесь my-org — пример. У вас должен быть свой namespace или организация.

Шаг 8. Опциональный деплой на VPS

Минимальная идея деплоя:

- pipeline подключается по SSH;
- сервер делает `docker pull`;
- затем выполняет `docker compose up -d`.

Пример логики:

```
docker pull ghcr.io/my-org/techip-tasks:<tag>
docker compose up -d
```

Для учебной практики этого достаточно на уровне понимания.

Типичные ошибки

Ошибка 1. Проект не собирается локально, но вы пытаетесь настраивать CI

Сначала проект должен работать локально.

Ошибка 2. Неправильный `working-directory`

Это особенно часто возникает в multi-service репозиториях.

Ошибка 3. `Docker build` не находит `Dockerfile`

Нужно проверить путь и build context.

Ошибка 4. Секреты записаны прямо в YAML

Это считается ошибкой постановки pipeline.

Ошибка 5. Pipeline запускается, но не проходит из-за отсутствия тестов

Если тестов пока нет, можно временно ограничиться `go build`, но в отчёте нужно это объяснить. Лучше всё же иметь хотя бы минимальные тесты.

Требования к отчёту

В отчёте должны быть представлены:

- тема и цель практической работы;
- краткое объяснение, что такое CI и CD;
- структура pipeline;
- выбранная платформа: GitHub Actions или GitLab CI;
- полный YAML-файл pipeline;
- пояснение шагов pipeline;
- способ формирования тега образа;
- объяснение, где должны храниться секреты;
- скриншот или лог успешного выполнения pipeline;
- при наличии — описание публикации образа и деплоя.

Контрольные вопросы

1. Чем CI отличается от CD?
2. Почему pipeline должен запускать тесты?
3. Зачем нужен автоматический build?
4. Почему важно собирать Docker-образ в CI, а не только локально?
5. Что такое CI secrets?
6. Почему нельзя хранить токены и SSH-ключи в репозитории?
7. Для чего нужен тег Docker-образа?
8. Что делает job docker-build?
9. Почему в multi-service проекте важен working-directory?
10. Какие риски возникают при полностью автоматическом деплое?